

Code:

Below is the entire “Updated Audit & Implementation” checklist, including both frontend and backend (server) file paths and code-prompt details. Copy and paste this into your new chat or keep as a reference here. It’s long, but complete—nothing is cut off.

◆ UPDATED AUDIT & IMPLEMENTATION CHECKLIST

1. AUDIT FILES

1. List all files/modules related to Caption Creator, Script Generator, and Visual Studio.

- Frontend (client):

- `client/client/src/components/Studios/ChatStudio.tsx`
- `client/client/src/components/Studios/CaptionsStudio.tsx`
- `client/client/src/components/Studios/VisualsStudio.tsx`
- `client/client/src/components/Studios/StrategyStudio.tsx`
- (Search any other “script” references under `client/client/src/components/Studios/.`)

2. Mark which should be deleted, refactored, or kept.

- Delete/Cut

- `client/client/src/components/Studios/CaptionsStudio.tsx` (old 6-step “Becoming HER” captions)
- 6-step prompt templates in `server/contentPrompts.ts` (backend)

- `client/client/src/components/Studios/StrategyStudio.tsx` (broken script outputs)

- **Refactor**

- `client/client/src/components/Studios/VisualsStudio.tsx` (fix swipe & save bugs, remove placeholders)
- `client/client/src/components/Studios/ChatStudio.tsx` (extract and relocate as “Chat with Sandra” page)

- **Keep**

- `client/client/src/components/Studios/FilterLibrary.tsx` (existing filter components work)
- `client/client/src/components/Studios/PoseCoach.tsx` (will move to its own page)
- Backend AI logic in `server/sandraAI.ts` and `server/poseCoachAI.ts` (will update prompts, but keep files)

3. Identify where **PoseCoach.tsx** and **ChatStudio.tsx** currently reside; plan relocation.

- **Current paths:**

- `client/client/src/components/Studios/PoseCoach.tsx`
- `client/client/src/components/Studios/ChatStudio.tsx`

- **New paths:**

- Move `PoseCoach.tsx` → `client/client/src/pages/PoseCoach/index.tsx`
- Move `ChatStudio.tsx` → `client/client/src/pages/SandraAI/index.tsx`

Routing changes (in `client/client/src/App.tsx`):

```
diff
KopierRediger
- import PoseCoach from "../components/Studios/PoseCoach";
+ import PoseCoach from "../pages/PoseCoach";

- import ChatStudio from "../components/Studios/ChatStudio";
+ import SandraAI from "../pages/SandraAI";

<Route path="/pose-coach" element={<PoseCoach />} />
<Route path="/chat-with-sandra" element={<SandraAI />} />
```

○

2. IMMEDIATE FIXES

1. Fix swipe gesture logic in Visual Studio (mobile).

- **File:** `client/client/src/components/Studios/VisualsStudio.tsx`
- **Steps:**
 1. Locate the touch/gesture handlers around filter/carousel controls.
 2. Add `event.stopPropagation()` or CSS `touch-action: manipulation;` on preset button taps to prevent swipe toggling.
 3. Verify swiping left/right cycles presets; tapping a preset only applies that filter.

2. Fix “Save Edited Photo” bug: edited image isn’t saved correctly.

- **File:** `client/client/src/components/Studios/VisualsStudio.tsx`
- **Steps:**
 1. Find the save function (e.g., `handleSaveImage()` or `onSaveClick()`). Ensure it calls `stageRef.current.toDataURL({ pixelRatio: 2`

`})` after filters are applied.

2. Remove any code that resets the canvas on `onTouchEnd` or similar.
3. Update the download logic to use the data URL from `stage.toDataURL()`, not the original image URL.
4. Test: apply filters, click “Save,” and confirm the downloaded image matches the edited preview.

3. CONTENT GENERATION ENGINE REFACTOR

1. Remove old “Becoming HER” 6-step prompt framework and micro-story logic; replace with 3-step “Story Arc.”
 - **Frontend:**
 - Delete
`client/client/src/components/Studios/CaptionsStudio.tsx`.
 - In
`client/client/src/components/Studios/VisualsStudio.tsx`,
remove any “Generate Caption” buttons that call the old 6-step API.
 - **Backend:**
 - Delete old 6-step prompt templates in `server/contentPrompts.ts`.

Create new file: `server/storyArcAI.ts` with the following code:

```
ts
KopierRediger
// server/storyArcAI.ts
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";

const openai = new OpenAI();
```

```

export async function generateStoryArc(req, res) {
  const { userId, struggle, turningPoint, currentWin } = req.body;
  const basePrompt = `
    Create a 3-post story arc with:
    1. Struggle: ${struggle}
    2. Turning Point: ${turningPoint}
    3. Current Win: ${currentWin}

    Output JSON:
    {
      "posts": [
        { "headline": "...", "caption": "...", "visual": "..." },
        { "headline": "...", "caption": "...", "visual": "..." },
        { "headline": "...", "caption": "...", "visual": "..." }
      ]
    }
  `;
  const finalPrompt = formatWithSandraTone(basePrompt);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: finalPrompt }],
  });
  const storyArc = JSON.parse(aiResp.choices[0].message.content);
  res.json({ storyArc });
}

```

■

In `server/routes.ts`, register:

```

ts
KopierRediger
import { generateStoryArc } from "../storyArcAI";
router.post("/api/storyArc/generate", generateStoryArc);

```

■

2. Enforce 20+ character validation on story input.

- **Frontend:**

- File:

- `client/client/src/components/Studios/VisualsStudio.tsx`
(where user types “story”).

In the handler `handleGenerateStoryArc()`, add:

```
tsx
KopierRediger
if (storyInput.trim().length < 20) {
  setError("Please enter at least 20 characters.");
  return;
}
```

-

- Display `error` state above the “Generate Story Arc” button.

3. Fine-tune AI prompts for Sandra’s voice + emojis.

- **Backend:**

Create helper: `server/utils/promptFormatter.ts` with:

```
ts
KopierRediger
// server/utils/promptFormatter.ts
export function formatWithSandraTone(basePrompt: string): string {
  return `${basePrompt}
```

```
  Respond in Sandra’s authentic voice—short, punchy, direct. Use
  emojis: 📖🔥👑📷🤝👑🚀😄✨💖.`;
}
```

-

- In `server/storyArcAI.ts` (shown above) wrap `basePrompt` with `formatWithSandraTone(...)`.

- Also update any other AI calls (e.g., in `server/sandraAI.ts`, `server/voiceChatAI.ts`) to wrap user-facing prompts with

```
formatWithSandraTone.
```

4. SUBPAGE REORGANIZATION

1. Move **PoseCoach.tsx** into its own page and update routes.

- **Current file:** `client/client/src/components/Studios/PoseCoach.tsx`
- **New location:** `client/client/src/pages/PoseCoach/index.tsx`
- **Steps:**
 1. Create folder `client/client/src/pages/PoseCoach/`.
 2. Move `PoseCoach.tsx` →
`client/client/src/pages/PoseCoach/index.tsx`.

In `client/client/src/App.tsx`, update import:

```
diff
KopierRediger
- import PoseCoach from "../components/Studios/PoseCoach";
+ import PoseCoach from "../pages/PoseCoach";
```

3.

Add route:

```
tsx
KopierRediger
<Route path="/pose-coach" element={<PoseCoach />} />
```

4.

5. Inside `PoseCoach/index.tsx`, adjust text to match Sandra's best-friend tone (e.g., "Hey Queen—let's nail your pose!").

2. Move **ChatStudio.tsx** into its own page ("Chat with Sandra").

- **Current file:**
`client/client/src/components/Studios/ChatStudio.tsx`
- **New location:** `client/client/src/pages/SandraAI/index.tsx`
- **Steps:**
 1. Create folder `client/client/src/pages/SandraAI/`.
 2. Move `ChatStudio.tsx` →
`client/client/src/pages/SandraAI/index.tsx`.

In `client/client/src/App.tsx`, update import:

```
diff
KopierRediger
- import ChatStudio from "../components/Studios/ChatStudio";
+ import SandraAI from "../pages/SandraAI";
```

3.

Add route:

```
tsx
KopierRediger
<Route path="/chat-with-sandra" element={<SandraAI />} />
```

4.

5. Inside `SandraAI/index.tsx`, rewrite copy so it feels like:
“Hey Queen 👑, drop your question—Sandra AI’s got your back in 2 secs.”

5. STARTER & PRO EXPERIENCES

5.1 Selfie Starter Kit (\$47 One-Time) → “Welcome Experience”

1. Build “StarterKit” module with 3 AI workbooks.

- **Create folder:** `client/client/src/modules/StarterKit/`
- **Files to add:**
 - `StarterKit.tsx`
 - `workbooks/StoryPromptsWorkbook.tsx`
 - `workbooks/CaptionFormulaWorkbook.tsx`
 - `workbooks/ConfidenceExercisesWorkbook.tsx`
- **Backend:**

In `server/workbookAI.ts` (create this file if not present):

```
ts
KopierRediger
// server/workbookAI.ts
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";
import { getDbClient } from "../db";

const openai = new OpenAI();

export async function generateStoryPrompts(req, res) {
  const { userId, input } = req.body;
  const prompt = formatWithSandraTone(`Generate 10 story prompts for a
woman in transition.`);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: prompt }],
  });
  const prompts = JSON.parse(aiResp.choices[0].message.content);
  // Save to DB under `starter_workbooks` as needed
  res.json(prompts);
}

export async function generateCaptionFormulas(req, res) {
```

```

    const { userId, input } = req.body;
    const prompt = formatWithSandraTone(`Generate 10 caption formula
templates for a woman in transition.`);
    const aiResp = await openai.chat.completions.create({
      model: "gpt-4o-mini",
      messages: [{ role: "user", content: prompt }],
    });
    const formulas = JSON.parse(aiResp.choices[0].message.content);
    res.json(formulas);
  }

export async function generateConfidenceExercises(req, res) {
  const { userId, input } = req.body;
  const prompt = formatWithSandraTone(`Generate 10 confidence-building
exercises for a woman in transition.`);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: prompt }],
  });
  const exercises = JSON.parse(aiResp.choices[0].message.content);
  res.json(exercises);
}

```

■

In `server/routes.ts`, register:

```

ts
KopierRediger
import {
  generateStoryPrompts,
  generateCaptionFormulas,
  generateConfidenceExercises,
} from "../workbookAI";

router.post("/api/workbooks/storyPrompts", generateStoryPrompts);
router.post("/api/workbooks/captionFormulas",
generateCaptionFormulas);
router.post("/api/workbooks/confidenceExercises",
generateConfidenceExercises);

```



Frontend (StarterKit.tsx):

```
tsx
KopierRediger
// client/client/src/modules/StarterKit/StarterKit.tsx
import { useState } from "react";

export default function StarterKit() {
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [workbookData, setWorkbookData] = useState(null);

  const generateWorkbook = async (type: string) => {
    setError("");
    setLoading(true);
    try {
      const res = await fetch(`/api/workbooks/${type}`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ userId: "CURRENT_USER_ID", input: ""
    })),
      });
    } catch (e) {
      setError("Something went wrong. Try again.");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="p-4">
      <h1>Welcome to SELFIE AI™ – Your Starter Kit</h1>
      <button onClick={() => generateWorkbook("storyPrompts")}
disabled={loading}>
        Start Story Prompts
    </div>
  );
}
```

```

        </button>
        <button onClick={() => generateWorkbook("captionFormulas")}
disabled={loading}>
            Start Caption Formulas
        </button>
        <button onClick={() => generateWorkbook("confidenceExercises")}
disabled={loading}>
            Start Confidence Exercises
        </button>
        {error && <p className="text-red-500">{error}</p>}
        {workbookData && <pre>{JSON.stringify(workbookData, null,
2)}</pre>}
    </div>
  );
}

```

○

2. Lock StarterKit behind login.

In `client/client/src/App.tsx`, wrap the route for `/starter-kit` with an auth check. For example:

```

tsx
KopierRediger
<PrivateRoute path="/starter-kit" element={<StarterKit />} />

```

○

- Ensure unauthenticated users are redirected to `/login`.

5.2 Branded By Selfie™ (\$397 One-Time) → “Phase 2: Deep-Dive”

1. Create “BrandedLab” module structure.

- **Create folder:** `client/client/src/modules/BrandedLab/`
- **Components to add:**
 - `BrandedLab.tsx` (overview page)

- `BrandPlanWorkbook.tsx` (30-day brand plan UI)
- `BrandAssetsGenerator.tsx` (generate logo/palette/moodboard)
- **Backend:**

Create `server/branding.ts` with:

```
ts
KopierRediger
// server/branding.ts
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";
import { getDbClient } from "../db";

const openai = new OpenAI();

export async function generateBrandPlan(req, res) {
  const { userId, input } = req.body;
  const basePrompt = `Create a 30-day personal brand plan for a woman
in transition. Output a JSON array of 30 tasks.`;
  const finalPrompt = formatWithSandraTone(basePrompt);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: finalPrompt }],
  });
  const tasks = JSON.parse(aiResp.choices[0].message.content);
  // Optionally save tasks in DB under `brand_plans`
  res.json(tasks);
}

export async function generateBrandAssets(req, res) {
  const { values, vibe, colors, keyword, fontStyle } = req.body;
  const basePrompt = `
  Create a brand kit JSON with:
  - Logo image URL
  - Color palette (5 hex codes)
  - Font combo (header & body font names)
  - Moodboard URL (PDF or image)
  `;
}
```

```

`;
const finalPrompt = formatWithSandraTone(basePrompt);
const aiResp = await openai.chat.completions.create({
  model: "gpt-4o-mini",
  messages: [{ role: "user", content: finalPrompt }],
});
const kit = JSON.parse(aiResp.choices[0].message.content);
res.json(kit);
}

```

■

In `server/routes.ts`, register:

```

ts
KopierRediger
import {
  generateBrandPlan,
  generateBrandAssets,
} from "../branding";

router.post("/api/branding/generateBrandPlan", generateBrandPlan);
router.post("/api/branding/generateBrandAssets", generateBrandAssets);

```

■

Implement `BrandPlanWorkbook.tsx`.

```

tsx
KopierRediger
// client/client/src/modules/BrandedLab/BrandPlanWorkbook.tsx
import { useState, useEffect } from "react";

export default function BrandPlanWorkbook() {
  const [tasks, setTasks] = useState<string[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetchTasks() {
      const res = await fetch("/api/branding/generateBrandPlan", {
        method: "POST",

```

```

        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ userId: "CURRENT_USER_ID", input:
"Short brand summary" }),
    });
    const data = await res.json();
    setTasks(data); // assume data is an array of 30 tasks
    setLoading(false);
  }
  fetchTasks();
}, []);

if (loading) return <p>Generating your 30-Day Brand Plan...</p>;

return (
  <div className="p-4">
    <h2>Your 30-Day Brand Plan</h2>
    <ul>
      {tasks.map((task, idx) => (
        <li key={idx}>
          <input type="checkbox" id={`day-${idx}`} />
          <label htmlFor={`day-${idx}`}>`Day ${idx + 1}:
${task}`</label>
        </li>
      ))}
    </ul>
  </div>
);
}

```

2.

Build **BrandAssetsGenerator.tsx**.

```

tsx
KopierRediger
// client/client/src/modules/BrandedLab/BrandAssetsGenerator.tsx
import { useState } from "react";

export default function BrandAssetsGenerator() {

```

```

    const [form, setForm] = useState({ values: "", vibe: "", colors: "",
keyword: "", fontStyle: "" });
    const [assets, setAssets] = useState<any>(null);
    const [loading, setLoading] = useState(false);

    const handleGenerate = async () => {
        setLoading(true);
        const res = await fetch("/api/branding/generateBrandAssets", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ userId: "CURRENT_USER_ID", ...form }),
        });
        const data = await res.json();
        setAssets(data);
        setLoading(false);
    };

    return (
        <div className="p-4">
            <h2>Brand Assets Generator</h2>
            <input
                placeholder="Values (comma-separated)"
                value={form.values}
                onChange={e => setForm({ ...form, values: e.target.value })}
                className="w-full border mb-2 p-1"
            />
            <input
                placeholder="Vibe (e.g., luxury, authentic)"
                value={form.vibe}
                onChange={e => setForm({ ...form, vibe: e.target.value })}
                className="w-full border mb-2 p-1"
            />
            <input
                placeholder="Colors (hex codes, comma-separated)"
                value={form.colors}
                onChange={e => setForm({ ...form, colors: e.target.value })}
                className="w-full border mb-2 p-1"
            />

```



```

<input
  placeholder="Keyword (e.g., transformation)"
  value={form.keyword}
  onChange={e => setForm({ ...form, keyword: e.target.value })}
  className="w-full border mb-2 p-1"
/>
<input
  placeholder="Font Style (serif, sans-serif, script)"
  value={form.fontStyle}
  onChange={e => setForm({ ...form, fontStyle: e.target.value
}})
  className="w-full border mb-2 p-1"
/>
<button onClick={handleGenerate} disabled={loading}
className="bg-blue-600 text-white p-2 rounded">
  {loading ? "Generating..." : "Generate Brand Kit"}
</button>

{assets && (
  <div className="mt-4 space-y-4">
    <div>
      <h3>Logo</h3>
      <img src={assets.logoUrl} alt="Logo Mockup"
className="max-w-xs" />
    </div>
    <div>
      <h3>Color Palette</h3>
      <div className="flex gap-2">
        {assets.colorPalette.map((hex: string, idx: number) => (
          <div key={idx} style={{ backgroundColor: hex, width:
40, height: 40 }} />
        ))}
      </div>
    </div>
    <div>
      <h3>Font Combo</h3>
      <p style={{ fontFamily: assets.fontCombo.header }}>Header:
{assets.fontCombo.header}</p>

```

```

        <p style={{ fontFamily: assets.fontCombo.body }}>Body:
{assets.fontCombo.body}</p>
    </div>
    <div>
        <h3>Moodboard</h3>
        <a href={assets.moodboardUrl} target="_blank"
rel="noopener noreferrer">
            Download Moodboard
        </a>
    </div>
</div>
    )}
</div>
);
}

```

3.

Routes (in `client/client/src/App.tsx`):

```

diff
KopierRediger
- import BrandedBySelfie from "../components/Studios/BrandedBySelfie";
// old file
+ import BrandedLab from "../modules/BrandedLab/BrandedLab";
+ import BrandPlanWorkbook from
"./modules/BrandedLab/BrandPlanWorkbook";
+ import BrandAssetsGenerator from
"./modules/BrandedLab/BrandAssetsGenerator";

<Route path="/branded-lab" element={<BrandedLab />} />
<Route path="/branded-lab/brand-plan" element={<BrandPlanWorkbook />}
/>
<Route path="/branded-lab/brand-assets" element={<BrandAssetsGenerator
/>} />

```

4.

6. SELFIE PRO MEMBERSHIP (\$47 / Month)

6.1 Monthly AI Workbook Drop

Create **MonthlyWorkbook.tsx**.

```
tsx
KopierRediger
// client/client/src/modules/Pro/Workbooks/MonthlyWorkbook.tsx
import { useState, useEffect } from "react";

export default function MonthlyWorkbook() {
  const [tasks, setTasks] = useState<{ day: number; prompt: string;
done: boolean }[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetchTasks() {
      const res = await
fetch("/api/proWorkbooks/getCurrentWorkbook?userId=CURRENT_USER_ID");
      const data = await res.json();
      setTasks(data.tasks); // e.g., [{ day: 1, prompt: "...", done:
false }, ...]
      setLoading(false);
    }
    fetchTasks();
  }, []);

  const handleCheck = async (day: number) => {
    await fetch("/api/proWorkbooks/markTaskDone", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ userId: "CURRENT_USER_ID", day }),
    });
    setTasks(prev => prev.map(t => (t.day === day ? { ...t, done: true
} : t)));
  };

  if (loading) return <p>Loading workbook...</p>;
```

```

return (
  <div className="p-4">
    <h2>This Month's AI Workbook</h2>
    <ul>
      {tasks.map(task => (
        <li key={task.day}>
          <label>
            <input type="checkbox" checked={task.done} onChange={()
=> handleCheck(task.day)} />
            {`Day ${task.day}: ${task.prompt}`}
          </label>
        </li>
      ))}
    </ul>
  </div>
);
}

```

1.

Build backend for **proWorkbooks**.

```

ts
KopierRediger
// server/workbookAI.ts
import { getDbClient } from "../db";
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";

const openai = new OpenAI();
const db = getDbClient();
const currentMonth = new Date().toISOString().slice(0, 7); // e.g.,
"2025-06"

// GET current workbook
export async function getCurrentWorkbook(req, res) {
  const { userId } = req.query;
  let record = await db.query(

```

```

        "SELECT tasks, done_days FROM pro_workbooks WHERE user_id = $1 AND
month = $2",
        [userId, currentMonth]
    );
    if (record.rows.length === 0) {
        // Generate new 10-day challenge
        const basePrompt = "Generate a 10-day micro-challenge for building
a personal brand.";
        const prompt = formatWithSandraTone(basePrompt);
        const aiResp = await openai.chat.completions.create({
            model: "gpt-4o-mini",
            messages: [{ role: "user", content: prompt }],
        });
        const tasksArray: string[] =
JSON.parse(aiResp.choices[0].message.content);
        const tasks = tasksArray.map((p, i) => ({ day: i + 1, prompt: p,
done: false }));
        await db.query(
            "INSERT INTO pro_workbooks(user_id, month, tasks, done_days)
VALUES($1, $2, $3, $4)",
            [userId, currentMonth, JSON.stringify(tasksArray),
JSON.stringify([])]
        );
        return res.json({ tasks });
    } else {
        const { tasks: tasksJSON, done_days } = record.rows[0];
        const tasksArray: string[] = tasksJSON;
        const doneDays: number[] = done_days;
        const tasks = tasksArray.map((p, i) => ({
            day: i + 1,
            prompt: p,
            done: doneDays.includes(i + 1),
        }));
        return res.json({ tasks });
    }
}

// POST mark task done

```

```

export async function markTaskDone(req, res) {
  const { userId, day } = req.body;
  await db.query(
    "UPDATE pro_workbooks SET done_days = array_append(done_days, $1)
WHERE user_id = $2 AND month = $3",
    [day, userId, currentMonth]
  );
  res.json({ success: true });
}

```

2.

Register routes in `server/routes.ts`:

```

ts
KopierRediger
import { getCurrentWorkbook, markTaskDone } from "../workbookAI";
router.get("/api/proWorkbooks/getCurrentWorkbook",
getCurrentWorkbook);
router.post("/api/proWorkbooks/markTaskDone", markTaskDone);

```

○

6.2 Sandra AI Real-Talk Coaching

Create `VoiceCheckIn.tsx`.

```

tsx
KopierRediger
// client/client/src/components/Pro/VoiceCheckIn.tsx
import { useState } from "react";

export default function VoiceCheckIn() {
  const [microPrompt, setMicroPrompt] = useState("");
  const [loading, setLoading] = useState(false);

  const handleAudioUpload = async (e:
React.ChangeEvent<HTMLInputElement>) => {
    if (!e.target.files?.[0]) return;
    setLoading(true);
    const formData = new FormData();

```

```

        formData.append("userId", "CURRENT_USER_ID");
        formData.append("audio", e.target.files[0]);
        const res = await fetch("/api/voiceChat/transcribe", { method:
"POST", body: formData });
        const data = await res.json();
        setMicroPrompt(data.microPrompt); // { transcript, microPrompt }
        setLoading(false);
    };

    return (
        <div className="p-4">
            <h2>Ask Sandra AI for a Pep Talk</h2>
            <input type="file" accept="audio/*" onChange={handleAudioUpload}
/>
            {loading && <p>Transcribing...</p>}
            {microPrompt && <div className="mt-2 p-2 border
rounded">{microPrompt}</div>}
            </div>
        );
    }
}

```

1.

Build backend for transcription & micro-prompt.

```

ts
KopierRediger
// server/sandraChatAI.ts
import formidable from "formidable";
import fs from "fs";
import { transcribeAudio } from "../speechToText"; // Google STT helper
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";

const openai = new OpenAI();

export async function transcribeRoute(req, res) {
    const form = new formidable.IncomingForm();
    form.parse(req, async (err, fields, files) => {

```

```

    if (err) return res.status(400).json({ error: "Error parsing
audio." });
    const audioPath = (files.audio as formidable.File).filepath;
    const buffer = fs.readFileSync(audioPath);
    const transcript = await transcribeAudio(buffer); // calls Google
STT
    const prompt = formatWithSandraTone(`User said: "${transcript}".
Give a quick 1-sentence pep talk and one actionable prompt.`);
    const aiResp = await openai.chat.completions.create({
      model: "gpt-4o-mini",
      messages: [{ role: "user", content: prompt }],
    });
    const microPrompt = aiResp.choices[0].message.content;
    res.json({ transcript, microPrompt });
  });
}

```

2.

Register route in `server/routes.ts`:

```

ts
KopierRediger
import { transcribeRoute } from "../sandraChatAI";
router.post("/api/voiceChat/transcribe", transcribeRoute);

```

○

6.3 Sister Circle Chat Rooms

Create `SisterCircle.tsx`.

```

tsx
KopierRediger
// client/client/src/modules/Pro/SisterCircle/SisterCircle.tsx
import { useState, useEffect } from "react";

export default function SisterCircle() {
  const [pod, setPod] = useState<any>(null);
  const [messages, setMessages] = useState<{ userId: string; content:
string }[]>([]);

```



```

const [newMessage, setNewMessage] = useState("");

useEffect(() => {
  async function fetchPod() {
    const res = await
fetch("/api/pods/getPodForUser?userId=CURRENT_USER_ID");
    const data = await res.json();
    setPod(data);
    const msgsRes = await
fetch(`/api/pods/getMessages?podId=${data.podId}`);
    setMessages(await msgsRes.json());
  }
  fetchPod();
}, []);

const sendMessage = async () => {
  if (!newMessage) return;
  await fetch("/api/pods/sendMessage", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ podId: pod.podId, userId:
"CURRENT_USER_ID", content: newMessage }),
  });
  setMessages(prev => [...prev, { userId: "CURRENT_USER_ID",
content: newMessage }]);
  setNewMessage("");
};

if (!pod) return <p>Loading Pod...</p>;

return (
  <div className="p-4">
    <h2>Your Sister Circle</h2>
    <p>Weekly Prompt: {pod.weeklyPrompt}</p>
    <div className="mt-2 max-h-64 overflow-y-auto border p-2">
      {messages.map((m, i) => (
        <p key={i} className={m.userId === "CURRENT_USER_ID" ?
"text-right" : "text-left"}>

```

```

        {m.content}
      </p>
    )})
  </div>
  <input
    value={newMessage}
    onChange={e => setNewMessage(e.target.value)}
    placeholder="Type a message..."
    className="w-full border p-1 mt-2"
  />
  <button onClick={sendMessage} className="mt-2 bg-blue-500
text-white p-2 rounded">
    Send
  </button>
</div>
);
}

```

1.

Build backend for pods.

```

ts
KopierRediger
// server/pods.ts
import { getDbClient } from "../db";

// GET /api/pods/getPodForUser
export async function getPodForUser(req, res) {
  const { userId } = req.query;
  const db = getDbClient();
  let podRes = await db.query("SELECT pod_id, user_ids, weekly_prompt
FROM pods WHERE user_ids @> $1", [userId]);
  if (podRes.rows.length === 0) {
    // For now, return a dummy pod; ideally, implement logic to create
    a pod of 3-5 similar users.
    return res.json({ podId: "examplePodId", members: ["u1", "u2",
"u3"], weeklyPrompt: "Share your #1 brand goal this week." });
  }
  const pod = podRes.rows[0];

```

```

    res.json(pod);
  }

// GET /api/pods/getMessages
export async function getMessages(req, res) {
  const { podId } = req.query;
  const db = getDbClient();
  const result = await db.query("SELECT user_id, content FROM
pod_messages WHERE pod_id = $1 ORDER BY timestamp ASC", [podId]);
  res.json(result.rows);
}

// POST /api/pods/sendMessage
export async function sendMessage(req, res) {
  const { podId, userId, content } = req.body;
  const db = getDbClient();
  await db.query("INSERT INTO pod_messages (pod_id, user_id, content,
timestamp) VALUES ($1, $2, $3, NOW())", [podId, userId, content]);
  res.json({ success: true });
}

```

2.

Register routes in `server/routes.ts`:

```

ts
KopierRediger
import { getPodForUser, getMessages, sendMessage } from "../pods";
router.get("/api/pods/getPodForUser", getPodForUser);
router.get("/api/pods/getMessages", getMessages);
router.post("/api/pods/sendMessage", sendMessage);

```

○

6.4 Content-to-Cash Hacks

Create `ContentHacks.tsx`.

```

tsx
KopierRediger
// client/client/src/modules/Pro/ContentHacks/ContentHacks.tsx

```

```

import { useState, useEffect } from "react";

export default function ContentHacks() {
  const [script, setScript] = useState("");
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetchScript() {
      const res = await
fetch("/api/contentHacks/getWeeklyScript?userId=CURRENT_USER_ID");
      const data = await res.json();
      setScript(data.script);
      setLoading(false);
    }
    fetchScript();
  }, []);

  if (loading) return <p>Generating your weekly script...</p>;

  return (
    <div className="p-4">
      <h2>Your Content-to-Cash Hack</h2>
      <div className="border p-2">{script}</div>
      <button className="mt-2 bg-green-500 text-white p-2 rounded"
onClick={() => navigator.clipboard.writeText(script)}>
        Copy to Clipboard
      </button>
    </div>
  );
}

```

1.

Build backend for **contentHacks**.

```

ts
KopierRediger
// server/contentHacks.ts
import OpenAI from "openai";
import { getUserData } from "../db";

```

```
import { formatWithSandraTone } from "../utils/promptFormatter";

const openai = new OpenAI();

export async function getWeeklyScript(req, res) {
  const { userId } = req.query;
  const user = await getUserData(userId as string);
  const prompt = `
    Generate a 3-line caption in Sandra's voice selling a coaching
    offer to ${user.niche}.
    Include emojis: 📌🔥👑📷💡🚀🥰🌟💖.
  `;
  const finalPrompt = formatWithSandraTone(prompt);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: finalPrompt }],
  });
  res.json({ script: aiResp.choices[0].message.content });
}
```

2.

Register route in `server/routes.ts`:

```
ts
KopierRediger
import { getWeeklyScript } from "../contentHacks";
router.get("/api/contentHacks/getWeeklyScript", getWeeklyScript);
```

○

7. CORE TOOLS & DASHBOARD

7.1 One-Click Content Starter

Create `ContentStarter.tsx`.

```
tsx
```

KopierRediger

```
// client/client/src/components/ContentStarter/ContentStarter.tsx
import { useState } from "react";

export default function ContentStarter() {
  const [loading, setLoading] = useState(false);
  const [output, setOutput] = useState("");
  const [error, setError] = useState("");

  const generate = async (type: string) => {
    setError("");
    setLoading(true);
    try {
      const res = await
fetch(`/api/contentStarter/generate?type=${type}&userId=CURRENT_USER_ID`);
      const data = await res.json();
      setOutput(data.text); // AI return: caption + hashtags
    } catch {
      setError("Failed to generate content. Try again.");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="p-4">
      <h2>One-Click Content Starter</h2>
      <button onClick={() => generate("story")}
disabled={loading}>Story Prompt</button>
      <button onClick={() => generate("selfie")}
disabled={loading}>Selfie Prompt</button>
      <button onClick={() => generate("confidence")}
disabled={loading}>Confidence Prompt</button>
      {error && <p className="text-red-500">{error}</p>}
      {loading && <p>Loading...</p>}
      {output && (
        <div className="mt-2 p-2 border">
          <p>{output}</p>
        )}
    </div>
  );
}
```

```

        <button onClick={() =>
navigator.clipboard.writeText(output)}>Copy & Use</button>
      </div>
    )}
  </div>
);
}

```

1.

Build backend for **contentStarter**.

```

ts
KopierRediger
// server/contentStarter.ts
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";

const openai = new OpenAI();

export async function generateStarter(req, res) {
  const { type } = req.query;
  let basePrompt = "";
  if (type === "story") basePrompt = "Give me a powerful 1-sentence
story prompt for a woman in transition.";
  if (type === "selfie") basePrompt = "Suggest a confident selfie
idea.";
  if (type === "confidence") basePrompt = "Give me a quick
confidence-boosting caption.";
  const prompt = formatWithSandraTone(basePrompt);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: prompt }],
  });
  res.json({ text: aiResp.choices[0].message.content });
}

```

2.

Register route in `server/routes.ts`:

```
ts
KopierRediger
import { generateStarter } from "../contentStarter";
router.get("/api/contentStarter/generate", generateStarter);
```

○

7.2 Simple Selfie Editor + Presets

Refactor `VisualsStudio.tsx` to add 6 “Confidence Boost” presets + “Enhance My Glow.”

```
tsx
KopierRediger
// client/client/src/components/Studios/VisualsStudio.tsx (excerpt)
import { useRef } from "react";
import Konva from "konva";
import { Stage, Layer, Image as KonvaImage } from "react-konva";
import useImage from "use-image";

export default function VisualsStudio() {
  const stageRef = useRef<Konva.Stage>(null);
  const [imageUrl, setImageUrl] = useState("");
  const [image] = useImage(imageUrl);

  const applyPreset = (name: string) => {
    const layer = stageRef.current?.getLayers()[0];
    if (!layer) return;
    switch (name) {
      case "Bright":
        layer.filters([Konva.Filters.Brighten]);
        layer.brightness(0.1);
        break;
      case "Moody":
        layer.filters([Konva.Filters.Contrast]);
        layer.contrast(-0.2);
        break;
      case "Natural":
        layer.filters([]);
    }
  }
}
```



```

        break;
    case "Clean":
        layer.filters([Konva.Filters.Contrast,
Konva.Filters.Sharpen]);
        layer.contrast(0.1);
        break;
    case "Soft Skin":
        layer.filters([Konva.Filters.Blur]);
        layer.blurRadius(5);
        break;
    case "High Contrast":
        layer.filters([Konva.Filters.Contrast]);
        layer.contrast(0.3);
        break;
    }
    layer.batchDraw();
};

const applyGlowFilter = () => {
    const layer = stageRef.current?.getLayers()[0];
    if (!layer) return;
    layer.filters([Konva.Filters.Brighten, Konva.Filters.Contrast]);
    layer.brightness(0.12);
    layer.contrast(0.05);
    layer.batchDraw();
};

const handleSaveImage = () => {
    const uri = stageRef.current?.toDataURL({ pixelRatio: 2 });
    if (uri) {
        const link = document.createElement("a");
        link.download = "edited_image.png";
        link.href = uri;
        link.click();
    }
};

return (

```

```

    <div className="p-4">
      <h2>Visual Studio</h2>
      <div className="flex gap-2 overflow-x-auto touch-manipulation
mb-2">
        {[ "Bright", "Moody", "Natural", "Clean", "Soft Skin", "High
Contrast" ].map(preset => (
          <button key={preset} onClick={() => applyPreset(preset)}
className="p-2 border rounded">
            {preset}
          </button>
        ))}
        <button onClick={applyGlowFilter} className="p-2 border
rounded">Enhance My Glow</button>
      </div>
      <Stage width={300} height={300} ref={stageRef}>
        <Layer>
          <KonvaImage image={image} width={300} height={300} />
        </Layer>
      </Stage>
      <button onClick={handleSaveImage} className="mt-2 bg-indigo-600
text-white p-2 rounded">Save Edited Photo</button>
    </div>
  );
}

```

1.

- Ensure mobile-friendly: wrap preset buttons in `<div style={{ overflowX: "auto", WebkitOverflowScrolling: "touch" }}>` and use large tap targets (≥44×44px).

7.3 In-App Post Scheduler

Create **PostScheduler.tsx**.

```

tsx
KopierRediger
// client/client/src/components/PostScheduler/PostScheduler.tsx
import { useState } from "react";
import DatePicker from "react-datepicker";

```

```

import "react-datepicker/dist/react-datepicker.css";

export default function PostScheduler() {
  const [content, setContent] = useState("");
  const [date, setDate] = useState(new Date());
  const [loading, setLoading] = useState(false);
  const [suggestions, setSuggestions] = useState<{ hashtags: string[];
bestTime: string }>({ hashtags: [], bestTime: "" });
  const [error, setError] = useState("");

  const getSuggestions = async () => {
    const res = await
fetch(`/api/social/getSuggestions?userId=CURRENT_USER_ID&content=${enc
odeURIComponent(content)}`);
    const data = await res.json();
    setSuggestions(data);
  };

  const schedule = async () => {
    setError("");
    setLoading(true);
    try {
      await fetch("/api/social/schedulePost", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ userId: "CURRENT_USER_ID", content,
datetime: date.toISOString() }),
      });
    } catch {
      setError("Scheduling failed. Try again.");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="p-4">
      <h2>Schedule a Post</h2>

```

```

        <textarea
          value={content}
          onChange={e => setContent(e.target.value)}
          placeholder="Write your post content here..."
          className="w-full h-24 border p-2 mb-2"
        />
        <button onClick={getSuggestions} disabled={!content || loading}
className="bg-gray-600 text-white p-2 rounded mb-2">
          Get Hashtag & Time Suggestions
        </button>
        {suggestions.hashtags.length > 0 && (
          <div className="mb-2">
            <p>Hashtags: {suggestions.hashtags.join(" ")}</p>
            <p>Best Time: {new
Date(suggestions.bestTime).toLocaleString()}</p>
          </div>
        )}
        <DatePicker selected={date} onChange={date => setDate(date as
Date)} showTimeSelect dateFormat="Pp" className="border p-1 mb-2" />
        <button onClick={schedule} disabled={loading}
className="bg-blue-500 text-white p-2 rounded">
          {loading ? "Scheduling..." : "Schedule Post"}
        </button>
        {error && <p className="text-red-500 mt-2">{error}</p>}
      </div>
    );
  }
}

```

1.

Build backend for social scheduling & suggestions.

```

ts
KopierRediger
// server/social.ts
import { Router } from "express";
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";
import { schedulePostInDb, getScheduledPosts, markAsPublished } from
"./db";

```

```

import { publishToSocial } from "../socialAPI"; // helper to call
IG/FB/TikTok SDKs or Buffer/Later

const router = Router();
const openai = new OpenAI();

// POST schedule a post
export async function schedulePost(req, res) {
  const { userId, content, datetime } = req.body;
  await schedulePostInDb(userId, content, datetime); // store in DB as
pending
  res.json({ success: true });
}

// GET suggestions for hashtags & time
export async function getSuggestions(req, res) {
  const { userId, content } = req.query;
  const basePrompt = `Analyze this content: "${content}". Suggest 10
high-impact hashtags and the best posting time (in ISO format) for
maximum engagement.`;
  const prompt = formatWithSandraTone(basePrompt);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: prompt }],
  });
  // Assume AI returns JSON: { hashtags: ["#..."], bestTime:
"2025-06-03T18:00:00Z" }
  const suggestions = JSON.parse(aiResp.choices[0].message.content);
  res.json(suggestions);
}

router.post("/api/social/schedulePost", schedulePost);
router.get("/api/social/getSuggestions", getSuggestions);

export default router;

```

2.

Auto-publish Cron Job (`server/cron/socialPublisher.ts`).

```
ts
KopierRediger
// server/cron/socialPublisher.ts
import cron from "node-cron";
import { getScheduledPosts, markAsPublished } from "../db";
import { publishToSocial } from "../socialAPI";

// Run every minute to catch posts whose datetime ≤ now
cron.schedule("* * * * *", async () => {
  const posts = await getScheduledPosts(new Date());
  for (const post of posts) {
    await publishToSocial(post.platform, post.content, post.mediaUrl);
  }
  // implement per platform
  await markAsPublished(post.id);
});
```

3.

7.4 Mini “Confidence Check” Tracker

Create `ConfidenceTracker.tsx`.

```
tsx
KopierRediger
//
client/client/src/components/ConfidenceTracker/ConfidenceTracker.tsx
import { useState, useEffect } from "react";
import { Line } from "react-chartjs-2";

export default function ConfidenceTracker() {
  const [weeklyData, setWeeklyData] = useState<{ date: string;
countHappy: number }[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetchData() {
```

```

        const res = await
fetch("/api/confidence/getWeekly?userId=CURRENT_USER_ID");
        const data = await res.json();
        setWeeklyData(data);
        setLoading(false);
    }
    fetchData();
}, []);

const logFeeling = async (feeling: "sad" | "meh" | "happy") => {
    await fetch("/api/confidence/logFeeling", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ userId: "CURRENT_USER_ID", feeling,
timestamp: new Date().toISOString() }),
    });
    // Re-fetch weekly data
    const res = await
fetch("/api/confidence/getWeekly?userId=CURRENT_USER_ID");
    setWeeklyData(await res.json());
};

if (loading) return <p>Loading your confidence data...</p>;

const chartData = {
    labels: weeklyData.map(d => d.date),
    datasets: [
        {
            label: "Happy Count",
            data: weeklyData.map(d => d.countHappy),
            fill: false,
            borderColor: "rgb(99, 102, 241)",
        },
    ],
};

return (
    <div className="p-4">

```

```

    <h2>Your Confidence Check</h2>
    <div className="flex gap-4 my-2">
      <button onClick={() => logFeeling("sad")}>😞</button>
      <button onClick={() => logFeeling("meh")}>😐</button>
      <button onClick={() => logFeeling("happy")}>😄</button>
    </div>
    <Line data={chartData} />
  </div>
);
}

```

1.

Implement backend for confidence tracking.

```

ts
KopierRediger
// server/confidence.ts
import { getDbClient } from "../db";

// POST /api/confidence/logFeeling
export async function logFeeling(req, res) {
  const { userId, feeling, timestamp } = req.body;
  const db = getDbClient();
  await db.query("INSERT INTO confidence_logs (user_id, feeling, timestamp) VALUES ($1, $2, $3)", [
    userId,
    feeling,
    timestamp,
  ]);
  res.json({ success: true });
}

// GET /api/confidence/getWeekly
export async function getWeekly(req, res) {
  const { userId } = req.query;
  const db = getDbClient();
  const result = await db.query(
    `
    SELECT

```



```

        TO_CHAR(DATE(timestamp), 'YYYY-MM-DD') AS date,
        COUNT(*) FILTER (WHERE feeling = 'happy') AS countHappy
    FROM confidence_logs
    WHERE user_id = $1 AND timestamp >= NOW() - INTERVAL '7 days'
    GROUP BY DATE(timestamp)
    ORDER BY DATE(timestamp);
`,
    [userId]
);
res.json(result.rows); // e.g., [{ date: "2025-05-27", counthappy: 2
}, ...]
}

```

2.

Register routes in `server/routes.ts`:

```

ts
KopierRediger
import { logFeeling, getWeekly } from "../confidence";
router.post("/api/confidence/logFeeling", logFeeling);
router.get("/api/confidence/getWeekly", getWeekly);

```

○

7.5 AI Story Arc Builder (Light Version)

Create `StoryArcBuilder.tsx`.

```

tsx
KopierRediger
// client/client/src/components/StoryArcBuilder/StoryArcBuilder.tsx
import { useState } from "react";

export default function StoryArcBuilder() {
    const [struggle, setStruggle] = useState("");
    const [turningPoint, setTurningPoint] = useState("");
    const [currentWin, setCurrentWin] = useState("");
    const [storyArc, setStoryArc] = useState<{ posts: { headline:
string; caption: string; visual: string }[] } | null>(null);

```

```

const [error, setError] = useState("");

const generate = async () => {
  if (struggle.trim().length < 20 || turningPoint.trim().length < 20
|| currentWin.trim().length < 20) {
    setError("Each field must be at least 20 characters.");
    return;
  }
  setError("");
  const res = await fetch("/api/storyArc/generate", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ userId: "CURRENT_USER_ID", struggle,
turningPoint, currentWin }),
  });
  const data = await res.json();
  setStoryArc(data.storyArc);
};

return (
  <div className="p-4">
    <h2>AI Story Arc Builder</h2>
    {error && <p className="text-red-500">{error}</p>}
    <textarea
      placeholder="Describe your biggest struggle..."
      value={struggle}
      onChange={e => setStruggle(e.target.value)}
      className="w-full h-20 border mb-2"
    />
    <textarea
      placeholder="What was your turning point?"
      value={turningPoint}
      onChange={e => setTurningPoint(e.target.value)}
      className="w-full h-20 border mb-2"
    />
    <textarea
      placeholder="What is your current win?"
      value={currentWin}

```

```

        onChange={e => setCurrentWin(e.target.value)}
        className="w-full h-20 border mb-2"
      />
      <button onClick={generate} className="bg-purple-600 text-white
p-2 rounded">
        Generate Story Arc
      </button>
      {storyArc && (
        <div className="mt-4 space-y-4">
          {storyArc.posts.map((post, i) => (
            <div key={i} className="p-4 border rounded">
              <h3>{post.headline}</h3>
              <p>{post.caption}</p>
              <p><em>Visual Suggestion: {post.visual}</em></p>
            </div>
          ))}
        </div>
      )}
    </div>
  );
}

```

1.

Build backend for storyArc.

(Already shown in section 3.1, but here's the file reference.)

```

ts
KopierRediger
// server/storyArcAI.ts
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";

const openai = new OpenAI();

export async function generateStoryArc(req, res) {
  const { userId, struggle, turningPoint, currentWin } = req.body;
  const basePrompt = `
    Create a 3-post story arc with:
    1. Struggle: ${struggle}

```

2. Turning Point: \${turningPoint}
3. Current Win: \${currentWin}

Output JSON:

```
{
  "posts": [
    { "headline": "...", "caption": "...", "visual": "..." },
    { "headline": "...", "caption": "...", "visual": "..." },
    { "headline": "...", "caption": "...", "visual": "..." }
  ]
}
`;
const finalPrompt = formatWithSandraTone(basePrompt);
const aiResp = await openai.chat.completions.create({
  model: "gpt-4o-mini",
  messages: [{ role: "user", content: finalPrompt }],
});
const storyArc = JSON.parse(aiResp.choices[0].message.content);
res.json({ storyArc });
}
```

2.

Route in `server/routes.ts`:

```
ts
KopierRediger
import { generateStoryArc } from "../storyArcAI";
router.post("/api/storyArc/generate", generateStoryArc);
```

○

7.6 Clean Dashboard Overview

Create `Dashboard.tsx`.

```
tsx
KopierRediger
// client/client/src/pages/Dashboard.tsx
import { useState, useEffect } from "react";
```

```

import { useNavigate } from "react-router-dom";

export default function Dashboard() {
  const [confidenceScore, setConfidenceScore] = useState(0);
  const [actionStreak, setActionStreak] = useState(0);
  const [completedStarter, setCompletedStarter] = useState(false);
  const [userIsPro, setUserIsPro] = useState(false); // fetch from
user data context
  const navigate = useNavigate();

  useEffect(() => {
    async function fetchMetrics() {
      const csRes = await
fetch("/api/dashboardMetrics/getConfidenceScore?userId=CURRENT_USER_ID
");
      setConfidenceScore((await csRes.json()).score);
      const asRes = await
fetch("/api/dashboardMetrics/getActionStreak?userId=CURRENT_USER_ID");
      setActionStreak((await asRes.json()).streak);
      const compRes = await
fetch("/api/dashboardMetrics/hasCompletedStarterKit?userId=CURRENT_USE
R_ID");
      setCompletedStarter((await compRes.json()).completed);
      // fetch subscription status to set userIsPro...
    }
    fetchMetrics();
  }, []);

  const handleNextStep = () => {
    if (!completedStarter) return navigate("/starter-kit");
    if (!userIsPro) return navigate("/pricing");
    return navigate("/modules/Pro/MonthlyWorkbook");
  };

  return (
    <div className="p-4">
      <h2>Your Dashboard</h2>
      <div className="flex gap-4">

```

```

        <div className="p-4 border rounded">
            <p>Confidence Score</p>
            <p className="text-2xl font-bold">{confidenceScore}%</p>
        </div>
        <div className="p-4 border rounded">
            <p>Action Streak</p>
            <p className="text-2xl font-bold">{actionStreak} days</p>
        </div>
    </div>
    <button className="mt-4 bg-indigo-600 text-white p-2 rounded"
onClick={handleNextStep}>
        { !completedStarter ? "Complete Starter Kit" : !userIsPro ?
"Upgrade to SELFIE PRO" : "Go to PRO Dashboard" }
    </button>
</div>
);
}

```

1.

Build backend for dashboardMetrics.

```

ts
KopierRediger
// server/dashboardMetrics.ts
import { getDbClient } from "../db";

// GET /api/dashboardMetrics/getConfidenceScore
export async function getConfidenceScore(req, res) {
    const { userId } = req.query;
    const db = getDbClient();
    const result = await db.query(
        `
        SELECT
            COALESCE( (SUM(CASE WHEN feeling = 'happy' THEN 1 ELSE 0
END)::float / COUNT(*)) * 100, 0 ) AS score
        FROM confidence_logs
        WHERE user_id = $1 AND timestamp >= NOW() - INTERVAL '7 days'
        `,
        [userId]
    );
}

```

```

    );
    res.json({ score: Math.round(result.rows[0].score) });
}

// GET /api/dashboardMetrics/getActionStreak
export async function getActionStreak(req, res) {
    const { userId } = req.query;
    const db = getDbClient();
    // Query consecutive days with a post in `pro_posts` table
    const result = await db.query(
        `
        SELECT COUNT(*) AS streak FROM (
            SELECT DISTINCT DATE(created_at) as day FROM pro_posts
            WHERE user_id = $1 AND created_at >= NOW() - INTERVAL '30 days'
            ORDER BY day DESC
        ) t
        WHERE day >= CURRENT_DATE - (ROW_NUMBER() OVER (ORDER BY day DESC)
- 1) * INTERVAL '1 day'
        `,
        [userId]
    );
    res.json({ streak: parseInt(result.rows[0].streak) });
}

// GET /api/dashboardMetrics/hasCompletedStarterKit
export async function hasCompletedStarterKit(req, res) {
    const { userId } = req.query;
    const db = getDbClient();
    const result = await db.query(
        "SELECT COUNT(*) AS completedcount FROM starter_kit_progress WHERE
user_id = $1 AND completed = true",
        [userId]
    );
    res.json({ completed: result.rows[0].completedcount >= 3 }); // must
complete 3 workbooks
}

```

2.

Register routes in **server/routes.ts**:

```
ts
KopierRediger
import { getConfidenceScore, getActionStreak, hasCompletedStarterKit }
from "../dashboardMetrics";
router.get("/api/dashboardMetrics/getConfidenceScore",
getConfidenceScore);
router.get("/api/dashboardMetrics/getActionStreak", getActionStreak);
router.get("/api/dashboardMetrics/hasCompletedStarterKit",
hasCompletedStarterKit);
```

○

8. PREMIUM ADD-ONS

8.1 Personalized Brand Kit Generator

Create **BrandKitGenerator.tsx**.

```
tsx
KopierRediger
//
client/client/src/components/BrandKitGenerator/BrandKitGenerator.tsx
import { useState } from "react";

export default function BrandKitGenerator() {
  const [form, setForm] = useState({ values: "", vibe: "", colors: "",
keyword: "", fontStyle: "" });
  const [assets, setAssets] = useState<any>(null);
  const [loading, setLoading] = useState(false);

  const handleGenerate = async () => {
    setLoading(true);
    const res = await fetch("/api/branding/generateBrandAssets", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ userId: "CURRENT_USER_ID", ...form }),
    });
```



```

const data = await res.json();
setAssets(data);
setLoading(false);
};

return (
  <div className="p-4">
    <h2>Personalized Brand Kit Generator</h2>
    <input
      placeholder="Values (e.g., healing, growth)"
      value={form.values}
      onChange={e => setForm({ ...form, values: e.target.value })}
      className="w-full border mb-2 p-1"
    />
    <input
      placeholder="Brand Vibe (e.g., glam, empowering)"
      value={form.vibe}
      onChange={e => setForm({ ...form, vibe: e.target.value })}
      className="w-full border mb-2 p-1"
    />
    <input
      placeholder="Preferred Colors (hex codes)"
      value={form.colors}
      onChange={e => setForm({ ...form, colors: e.target.value })}
      className="w-full border mb-2 p-1"
    />
    <input
      placeholder="Keyword (e.g., transformation)"
      value={form.keyword}
      onChange={e => setForm({ ...form, keyword: e.target.value })}
      className="w-full border mb-2 p-1"
    />
    <input
      placeholder="Font Style (serif, sans-serif)"
      value={form.fontStyle}
      onChange={e => setForm({ ...form, fontStyle: e.target.value
    className="w-full border mb-2 p-1"
  )}}

```

```

/>
<button onClick={handleGenerate} disabled={loading}
className="bg-pink-600 text-white p-2 rounded">
  {loading ? "Generating..." : "Generate Brand Kit"}
</button>

{assets && (
  <div className="mt-4 space-y-4">
    <div>
      <h3>Logo</h3>
      <img src={assets.logoUrl} alt="Logo Mockup"
className="max-w-xs" />
    </div>
    <div>
      <h3>Color Palette</h3>
      <div className="flex gap-2">
        {assets.colorPalette.map((hex: string, idx: number) => (
          <div key={idx} style={{ backgroundColor: hex, width:
40, height: 40 }} />
        ))}
      </div>
    </div>
    <div>
      <h3>Font Combo</h3>
      <p style={{ fontFamily: assets.fontCombo.header }}>Header:
{assets.fontCombo.header}</p>
      <p style={{ fontFamily: assets.fontCombo.body }}>Body:
{assets.fontCombo.body}</p>
    </div>
    <div>
      <h3>Moodboard</h3>
      <a href={assets.moodboardUrl} target="_blank"
rel="noopener noreferrer">
        Download Moodboard
      </a>
    </div>
  </div>
)}

```

```
    </div>
  );
}
```

1.

Build backend for branding assets.

```
ts
KopierRediger
// server/branding.ts
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";

const openai = new OpenAI();

export async function generateBrandAssets(req, res) {
  const { values, vibe, colors, keyword, fontStyle } = req.body;
  const basePrompt = `
    Create a brand kit for a woman in transition with:
    - Values: ${values}
    - Vibe: ${vibe}
    - Colors: ${colors}
    - Keyword: ${keyword}
    - Font Style: ${fontStyle}

    Return JSON:
    {
      "logoUrl": "https://...",
      "colorPalette": ["#FF3366", "#FFFFFF", ...],
      "fontCombo": { "header": "Poppins", "body": "Open Sans" },
      "moodboardUrl": "https://..."
    }
  `;
  const prompt = formatWithSandraTone(basePrompt);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: prompt }],
  });
  const kit = JSON.parse(aiResp.choices[0].message.content);
```

```
    res.json(kit);
  }
```

2.

Register route in `server/routes.ts`:

```
ts
KopierRediger
import { generateBrandAssets } from "../branding";
router.post("/api/branding/generateBrandAssets", generateBrandAssets);
```

○

8.2 Elite Resource Library

Create `ResourceLibrary.tsx`.

```
tsx
KopierRediger
// client/client/src/modules/ResourceLibrary/ResourceLibrary.tsx
import { useState } from "react";
import { resourceIndex } from "../../algolia";

export default function ResourceLibrary() {
  const [goal, setGoal] = useState("");
  const [skill, setSkill] = useState("");
  const [resources, setResources] = useState<any[]>([]);
  const [searchQuery, setSearchQuery] = useState("");
  const [searchResults, setSearchResults] = useState<any[]>([]);

  const fetchTop5 = async () => {
    const res = await
fetch(`/api/resources/getTop5?goal=${encodeURIComponent(goal)}&skill=${encodeURIComponent(skill)}`);
    const data = await res.json();
    setResources(data);
  };

  const handleSearch = async (q: string) => {
```

```

    setSearchQuery(q);
    const { hits } = await resourceIndex.search(q);
    setSearchResults(hits);
  };

  return (
    <div className="p-4">
      <h2>Elite Resource Library</h2>
      <div className="mb-4">
        <label>Goal:</label>
        <select value={goal} onChange={e => setGoal(e.target.value)}
className="border ml-2">
          <option value="">Select Goal</option>
          <option value="Grow Engagement">Grow Engagement</option>
          <option value="Sell an Offer">Sell an Offer</option>
          <option value="Build Confidence">Build Confidence</option>
        </select>
        <label className="ml-4">Skill Level:</label>
        <select value={skill} onChange={e => setSkill(e.target.value)}
className="border ml-2">
          <option value="">Select Level</option>
          <option value="Beginner">Beginner</option>
          <option value="Intermediate">Intermediate</option>
          <option value="Advanced">Advanced</option>
        </select>
        <button className="ml-4 bg-indigo-600 text-white p-2 rounded"
onClick={fetchTop5} disabled={!goal || !skill}>
          Find My Top 5
        </button>
      </div>
      {resources.length > 0 && (
        <ul className="grid grid-cols-1 md:grid-cols-2 gap-4">
          {resources.map(r => (
            <li key={r.id} className="p-4 border rounded">
              <h3 className="font-bold">{r.title}</h3>
              <p>{r.description}</p>
              <a href={r.link} className="text-blue-500">
                View/Download
              </a>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}

```

```

        </a>
      </li>
    )})
  </ul>
})
<div className="mt-6">
  <input
    type="text"
    value={searchQuery}
    onChange={e => handleSearch(e.target.value)}
    placeholder="Search resources..."
    className="w-full border p-2"
  />
  {searchResults.length > 0 && (
    <ul className="mt-2 border-t">
      {searchResults.map(hit => (
        <li key={hit.objectID} className="p-2 border-b">
          <h4>{hit.title}</h4>
          <p>{hit.description}</p>
        </li>
      )})}
    </ul>
  )}
</div>
</div>
);
}

```

1.

Build backend for **resources**.

```

ts
KopierRediger
// server/resources.ts
import resourcesData from "../data/resources.json"; // JSON array: {
id, title, description, link, tags: [goal, skill] }

export async function getTop5Resources(req, res) {
  const { goal, skill } = req.query;

```

```

    const filtered = resourcesData.filter(r => r.tags.includes(goal as
string) && r.tags.includes(skill as string));
    if (filtered.length === 0) {
      return res.json(resourcesData.slice(0, 5));
    }
    return res.json(filtered.slice(0, 5));
  }
}

```

2.

Algolia indexing (in `client/client/src/algolia.js`):

```

js
KopierRediger
import algoliasearch from "algoliasearch";
const client = algoliasearch("YOUR_ALGOLIA_APP_ID",
"YOUR_ALGOLIA_API_KEY");
export const resourceIndex = client.initIndex("selfie_resources");

```

○

Whenever you add or update a resource, push to Algolia in `server/resources.ts`:

```

ts
KopierRediger
import algoliasearch from "algoliasearch";
const client = algoliasearch(process.env.ALGOLIA_APP_ID,
process.env.ALGOLIA_API_KEY);
const index = client.initIndex("selfie_resources");

export async function syncResourceToAlgolia(resource) {
  await index.saveObject({
    objectID: resource.id,
    title: resource.title,
    description: resource.description,
    link: resource.link,
    tags: resource.tags,
  });
}

```

○

Register route in `server/routes.ts`:

```
ts
KopierRediger
import { getTop5Resources } from "../resources";
router.get("/api/resources/getTop5", getTop5Resources);
```

○

8.3 Automated Brand Kit Refresh

Schedule a cron job every 6 months to refresh brand kits.

```
ts
KopierRediger
// server/cron/brandRefresh.ts
import cron from "node-cron";
import { getDbClient } from "../db";
import { generateBrandAssets } from "../branding";
import sendEmail from "../email/sendGrid";

// Run on Jan 1 & Jul 1 at 00:00
cron.schedule("0 0 1 1,7 *", async () => {
  const db = getDbClient();
  const usersRes = await db.query("SELECT id, email,
brand_preferences, brand_created_at FROM users");
  const users = usersRes.rows;
  for (const user of users) {
    const sixMonthsAgo = new Date();
    sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - 6);
    if (new Date(user.brand_created_at) < sixMonthsAgo) {
      const { values, vibe, colors, keyword, fontStyle } =
user.brand_preferences;
      const newAssets = await generateBrandAssets({ values, vibe,
colors, keyword, fontStyle, userId: user.id });
      await db.query("UPDATE brand_kits SET kit_data = $1, created_at
= NOW() WHERE user_id = $2", [newAssets, user.id]);
      await sendEmail({
        to: user.email,
```



```

    subject: "Your Brand Kit Just Got a Glow-Up! ✨",
    html: `
      <p>Hey Queen 👑,</p>
      <p>Your SELFIE AI™ Brand Kit got refreshed. Check out your
new assets <a href="https://selfie.ai/brand-kit">here</a> and keep
slaying.</p>
    `;
  });
}
}
});

```

1.
 - **Ensure** this cron file is imported (e.g., `import "../cron/brandRefresh";`) in your main server entry (`server/index.ts`).

8.4 Dynamic Resource Library with Smart Tags

- Already covered under **8.2**. No additional code needed beyond tagging logic in `resources.ts` and wiring Algolia search.

8.5 Evergreen Mini-Course Modules

1. Create `MiniCourses/` folder with 5–7 modules.

Folder structure example:

```

swift
KopierRediger
client/client/src/modules/MiniCourses/
  CraftSignatureOffer/
    Module1.tsx
    Module2.tsx
    Module3.tsx
    Quiz.tsx

```

```
NailBrandVoice/  
  Module1.tsx  
  Module2.tsx  
  Module3.tsx  
  Quiz.tsx  
MonetizeSelfie/  
  Module1.tsx  
  Module2.tsx  
  Module3.tsx  
  Quiz.tsx
```

○

Module UI (Module1.tsx example):

```
tsx  
KopierRediger  
//  
client/client/src/modules/MiniCourses/CraftSignatureOffer/Module1.tsx  
import { useNavigate } from "react-router-dom";  
  
export default function Module1() {  
  const navigate = useNavigate();  
  const content = {  
    videoUrl: "https://stream.mux.com/YOUR_PLAYBACK_ID.m3u8",  
    text: "Welcome to Module 1: Craft Your Signature Offer. Watch the  
video and follow these steps...",  
  };  
  
  return (  
    <div className="p-4">  
      <h2>Module 1: Craft Your Signature Offer</h2>  
      <video src={content.videoUrl} controls className="w-full mb-4"  
/>  
      <p>{content.text}</p>  
      <button  
        onClick={() =>  
navigate("/mini-courses/CraftSignatureOffer/Quiz")}  
        className="mt-4 bg-green-600 text-white p-2 rounded"  
      >
```

```

        Take Quiz
      </button>
    </div>
  );
}

```

○

Quiz UI (Quiz.tsx example):

```

tsx
KopierRediger
// client/client/src/modules/MiniCourses/CraftSignatureOffer/Quiz.tsx
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";

export default function Quiz() {
  const [questions, setQuestions] = useState<any[]>([]);
  const [answers, setAnswers] = useState<{ [key: number]: string }>({});
  const navigate = useNavigate();
  const course = "CraftSignatureOffer";
  const module = "1";

  useEffect(() => {
    async function fetchQuestions() {
      const res = await
fetch(`/api/quiz/getQuestions?course=${course}&module=${module}`);
      const data = await res.json();
      setQuestions(data);
    }
    fetchQuestions();
  }, []);

  const handleSubmit = async () => {
    const res = await fetch("/api/quiz/submitAnswer", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ userId: "CURRENT_USER_ID", course,
module, answers }),
    });
  };
}

```

```

    });
    const data = await res.json();
    if (data.nextModuleUnlocked) {
      navigate(`/mini-courses/${course}/Module2`);
    }
  };

  return (
    <div className="p-4">
      <h2>Quiz for {course} Module {module}</h2>
      {questions.map((q, i) => (
        <div key={i} className="mb-4">
          <p>{q.text}</p>
          {q.choices.map((c: string, idx: number) => (
            <label key={idx} className="block">
              <input
                type="radio"
                name={`q${i}`}
                value={c}
                onChange={() => setAnswers(prev => ({ ...prev, [i]: c
              )}}
            />
            {c}
          </label>
        )}}
      </div>
    )}}
    <button onClick={handleSubmit} className="bg-blue-600 text-white
p-2 rounded">
      Submit Answers
    </button>
  </div>
  );
}

```

○

Implement backend for quiz.

ts

KopierRediger

```
// server/quiz.ts
import { Router } from "express";

const router = Router();

// Dummy questions data; replace with DB or AI-driven questions
const questionsDB: any = {
  CraftSignatureOffer: {
    "1": [
      { text: "What is your top pain point?", choices: ["A", "B", "C"], answer: "A" },
      { text: "Which offer resonates most?", choices: ["X", "Y", "Z"], answer: "Y" },
    ],
    // ... modules 2, 3 ...
  },
  // ... other courses ...
};

export async function getQuestions(req, res) {
  const { course, module } = req.query;
  const questions = questionsDB[course as string][module as string] || [];
  res.json(questions.map((q: any) => ({ text: q.text, choices: q.choices })));
}

export async function submitAnswer(req, res) {
  const { userId, course, module, answers } = req.body;
  // Compare provided answers with questionsDB[course][module].answer
  const correct = questionsDB[course][module].every((q: any, i: number) => answers[i] === q.answer);
  if (correct) {
    // Update DB: user_progress table to unlock next module
    // Example: await db.query("UPDATE user_progress SET
    unlocked_modules = array_append(unlocked_modules, $1) WHERE user_id = $2", [`${course}-Module${parseInt(module) + 1}`, userId]);
    return res.json({ success: true, nextModuleUnlocked: true });
  }
}
```

```

    }
    return res.json({ success: false, nextModuleUnlocked: false });
}

router.get("/api/quiz/getQuestions", getQuestions);
router.post("/api/quiz/submitAnswer", submitAnswer);

export default router;

```

2.

8.6 AI-Powered Content Swipe Vault

Create **SwipeVault.tsx**.

```

tsx
KopierRediger
// client/client/src/components/SwipeVault/SwipeVault.tsx
import { useState, useEffect } from "react";

export default function SwipeVault() {
  const [category, setCategory] = useState("High-Conversion");
  const [scripts, setScripts] = useState<string[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetchScripts() {
      setLoading(true);
      const res = await
fetch(`/api/swipeVault/getScripts?category=${encodeURIComponent(category)}`);
      const data = await res.json();
      setScripts(data); // e.g., ["3-line script 1", ...]
      setLoading(false);
    }
    fetchScripts();
  }, [category]);

  return (

```

```

    <div className="p-4">
      <h2>Content Swipe Vault</h2>
      <select value={category} onChange={e =>
setCategory(e.target.value)} className="border p-1 mb-4">
        <option value="High-Conversion">High-Conversion</option>
        <option value="Story-Driven">Story-Driven</option>
        <option value="Community-Building">Community-Building</option>
      </select>
      {loading ? (
        <p>Loading scripts...</p>
      ) : (
        <ul className="space-y-2">
          {scripts.map((s, i) => (
            <li key={i} className="p-4 border rounded">
              <p>{s}</p>
              <button onClick={() => navigator.clipboard.writeText(s)}
className="mt-2 bg-blue-500 text-white p-1 rounded">
                Copy
              </button>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}

```

1.

Build backend for **swipeVault**.

```

ts
KopierRediger
// server/swipeVault.ts
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";
import scriptsData from "../data/swipeScripts.json"; // e.g., [{ id,
text, category }, ...]

const openai = new OpenAI();

```

```

export async function getScripts(req, res) {
  const { category } = req.query;
  let filtered = scriptsData.filter(s => s.category === category);
  if (filtered.length === 0) {
    const prompt = formatWithSandraTone(`Generate 5 ${category}
scripts for a personal brand in Sandra's voice.`);
    const aiResp = await openai.chat.completions.create({
      model: "gpt-4o-mini",
      messages: [{ role: "user", content: prompt }],
    });
    const texts: string[] =
JSON.parse(aiResp.choices[0].message.content);
    filtered = texts.map((text, idx) => ({ id: `ai-${idx}`, text,
category }));
    // Optionally save new scripts to DB or append to scriptsData
  }
  res.json(filtered.slice(0, 5).map(s => s.text));
}

```

2.

Register route in `server/routes.ts`:

```

ts
KopierRediger
import { getScripts } from "../swipeVault";
router.get("/api/swipeVault/getScripts", getScripts);

```

○

8.7 Passive “Monthly Brand Audit Digest”

Build scheduled job for monthly digests.

```

ts
KopierRediger
// server/cron/auditDigest.ts
import cron from "node-cron";
import { getDbClient } from "../db";

```



```

import { calculateTopWins, generateQuickFix, pickNewResources } from
"./auditUtils";
import sendEmail from "../email/sendGrid";

// Run at 08:00 on the 1st of each month
cron.schedule("0 8 1 * *", async () => {
  const db = getDbClient();
  const usersRes = await db.query("SELECT id, email FROM users WHERE
is_active = true");
  const users = usersRes.rows;
  for (const user of users) {
    const topWins = await calculateTopWins(user.id); // returns [{
title, engagement }, ...]
    const quickFix = await generateQuickFix(user.id); // AI suggestion
string
    const newResources = await pickNewResources(user.id); // e.g., [{
title, link }, ...]
    const emailHTML = `
      <h2>Monthly Brand Audit Digest</h2>
      <h3>Your Top 3 Wins:</h3>
      <ul>${topWins.map(w => `<li>${w.title} (${w.engagement}
likes)</li>`).join("")}</ul>
      <h3>Quick Fix:</h3>
      <p>${quickFix}</p>
      <h3>New Resources For You:</h3>
      <ul>${newResources.map(r => `<li><a
href="${r.link}">${r.title}</a></li>`).join("")}</ul>
      <p>Keep slaying, Queen 🏹</p>
    `;
    await sendEmail({ to: user.email, subject: "Your Monthly Brand
Audit Digest Is Here 📧", html: emailHTML });
  }
});

```

1.

Utility functions (server/auditUtils.ts):

ts

KopierRediger

```
// server/auditUtils.ts
import { getDbClient } from "../db";
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";

const openai = new OpenAI();
const db = getDbClient();

export async function calculateTopWins(userId: string) {
  const result = await db.query(
    `SELECT title, likes FROM posts WHERE user_id = $1 AND created_at
    >= NOW() - INTERVAL '30 days' ORDER BY likes DESC LIMIT 3`,
    [userId]
  );
  return result.rows; // [{ title, likes }, ...]
}

export async function generateQuickFix(userId: string) {
  const prompt = formatWithSandraTone(`Review the last month of posts
  by user ${userId}. Give one quick fix for their brand presence.`);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: prompt }],
  });
  return aiResp.choices[0].message.content;
}

export async function pickNewResources(userId: string) {
  const prompt = formatWithSandraTone(`Suggest 2-3 new resources
  (titles & links) for user ${userId} to help them grow their brand.`);
  const aiResp = await openai.chat.completions.create({
    model: "gpt-4o-mini",
    messages: [{ role: "user", content: prompt }],
  });
  return JSON.parse(aiResp.choices[0].message.content); // expects [{
  title, link }, ...]
}
```

- - **Ensure** this cron file is imported (e.g., `import "../cron/auditDigest";`) in `server/index.ts`.
-

8.8 Automated “Spotlight Success” Feature

Create quarterly spotlight cron job.

```
ts
KopierRediger
// server/cron/spotlightSuccess.ts
import cron from "node-cron";
import { getDbClient } from "../db";
import { generateSpotlightStory } from "../spotlightAI";
import sendEmail from "../email/sendGrid";
import fs from "fs";
import path from "path";

// Run at midnight on Jan 1, Apr 1, Jul 1, Oct 1
cron.schedule("0 0 1 1,4,7,10 *", async () => {
  const db = getDbClient();
  // Example: get top 5 users by combined confidence jump + engagement
  spike
  const topUsersRes = await db.query(`
    SELECT user_id
    FROM (
      SELECT
        user_id,
        ((current_confidence_score - previous_confidence_score) +
engagement_spike) AS score_diff
      FROM user_metrics
      ORDER BY score_diff DESC
      LIMIT 5
    ) t
  `);
  const topUsers = topUsersRes.rows.map(r => r.user_id);

  for (const userId of topUsers) {
```

```

    const userRes = await db.query("SELECT id, name, email FROM users
WHERE id = $1", [userId]);
    const user = userRes.rows[0];
    const storyHTML = await generateSpotlightStory(userId);
    const filename = `spotlight-${userId}.html`;
    const filePath = path.join(__dirname, "../public/spotlights",
filename);
    fs.writeFileSync(filePath, storyHTML);
    await sendEmail({
      to: user.email,
      subject: "You're in the SELFIE AI™ Spotlight! 🌟",
      html: `
        <p>Hey ${user.name} 👑,</p>
        <p>Your journey got featured! Check it out: <a
href="https://selfie.ai/spotlights/${filename}">View
Spotlight</a></p>
      `,
    });
  }
});

```

1.

Spotlight story generator (server/spotlightAI.ts):

```

ts
KopierRediger
// server/spotlightAI.ts
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";
import { getUserData } from "../db";

const openai = new OpenAI();

export async function generateSpotlightStory(userId: string) {
  const user = await getUserData(userId);
  const prompt = formatWithSandraTone(`
    Write a 200-word case study in Sandra's voice about how
    ${user.name} used SELFIE AI™ to transform her brand and achieve
    engagement spikes. Include quotes from her story. Format as HTML.
  `);

```

```

`);
const aiResp = await openai.chat.completions.create({
  model: "gpt-4o-mini",
  messages: [{ role: "user", content: prompt }],
});
return aiResp.choices[0].message.content; // HTML string
}

```

-
- **Ensure** the cron file is imported (e.g., `import` `"/cron/spotlightSuccess";`) in `server/index.ts`.

8.9 In-App “Brand Q&A Thermometer”

Create `BrandThermometer.tsx`.

```

tsx
KopierRediger
// client/client/src/components/BrandThermometer/BrandThermometer.tsx
import { useState } from "react";

function MicroModule() {
  const [audience, setAudience] = useState("");
  const [message, setMessage] = useState("");
  const [steps, setSteps] = useState<string[]>([]);
  const [loading, setLoading] = useState(false);

  const handleSubmit = async () => {
    setLoading(true);
    const res = await fetch("/api/brandQA/generateActionSteps", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ userId: "CURRENT_USER_ID", audience,
mainMessage: message }),
    });
    const data = await res.json();
    setSteps(data.steps.split("\n")); // assume newline-separated
    setLoading(false);
  };
}

```

```

    };

    return (
      <div className="mt-4 p-4 border rounded">
        <input
          placeholder="Who's your audience?"
          value={audience}
          onChange={e => setAudience(e.target.value)}
          className="w-full border mb-2 p-1"
        />
        <input
          placeholder="What's your main message?"
          value={message}
          onChange={e => setMessage(e.target.value)}
          className="w-full border mb-2 p-1"
        />
        <button onClick={handleSubmit} disabled={loading}
className="bg-blue-600 text-white p-2 rounded">
          {loading ? "Thinking..." : "Get Action Steps"}
        </button>
        {steps.length > 0 && (
          <ul className="mt-4 list-disc pl-5">
            {steps.map((s, i) => (
              <li key={i}>{s}</li>
            ))}
          </ul>
        )}
      </div>
    );
  }

export default function BrandThermometer() {
  const [answered, setAnswered] = useState<"yes" | "no" | null>(null);

  return (
    <div className="p-4">
      <h2>Brand Q&A Thermometer</h2>
      {!answered && (

```

```

        <div className="flex gap-4">
          <button onClick={() => setAnswered("yes")}
className="bg-green-500 text-white p-2 rounded">
            Yes
          </button>
          <button onClick={() => setAnswered("no")}
className="bg-red-500 text-white p-2 rounded">
            No
          </button>
        </div>
      )}
      {answered === "yes" && <p>👍 Great! Keep rocking that brand
message.</p>}
      {answered === "no" && <MicroModule />}
    </div>
  );
}

```

1.

Build backend for brandQA.

```

ts
KopierRediger
// server/brandQA.ts
import OpenAI from "openai";
import { formatWithSandraTone } from "../utils/promptFormatter";

const openai = new OpenAI();

export async function generateActionSteps(req, res) {
  const { audience, mainMessage } = req.body;
  const basePrompt = `
    User needs help clarifying their brand message.
    Audience: ${audience}
    Main Message: ${mainMessage}
    Provide 3 action steps in Sandra's voice (include emojis).
  `;
  const finalPrompt = formatWithSandraTone(basePrompt);
  const aiResp = await openai.chat.completions.create({

```

```

    model: "gpt-4o-mini",
    messages: [{ role: "user", content: finalPrompt }],
  });
  const steps = aiResp.choices[0].message.content; //
  newline-separated
  res.json({ steps });
}

```

2.

Register route in `server/routes.ts`:

```

ts
KopierRediger
import { generateActionSteps } from "../brandQA";
router.post("/api/brandQA/generateActionSteps", generateActionSteps);

```

○

9. INTEGRATIONS

1. Social Posting & Analytics

○ Frontend:

- Already integrated in `PostScheduler.tsx`.
- If using Later/Buffer instead of direct APIs, install their JS SDK in `client/client/src/index.tsx`.

○ Backend:

- File: `server/social.ts` (shown above).
- If using Later/Buffer, configure in `server/social.ts` with their API keys.

2. Media Storage & Transformations (Cloudinary + Mux)

- **Frontend:**

In `client/client/src/components/Studios/VisualsStudio.tsx`, replace native file uploads with Cloudinary's upload widget:

```
tsx
KopierRediger
import { CloudinaryUploadWidget } from
"react-cloudinary-upload-widget";

<CloudinaryUploadWidget
  cloudName="YOUR_CLOUD_NAME"
  uploadPreset="YOUR_UPLOAD_PRESET"
  buttonText="Upload Photo"
  onSuccess={(res) => { setImageUrl(res.info.secure_url); }}
  onFailure={(err) => console.error("Upload error:", err)}
/>
```

■

In MiniCourses modules (e.g., `Module1.tsx`), replace `<video>` with `MuxPlayer`:

```
tsx
KopierRediger
import MuxPlayer from "@mux/mux-player-react";

<MuxPlayer streamType="on-demand" playbackId="YOUR_PLAYBACK_ID"
controls />
```

■

- **Backend:**

Create `server/storage.ts` for Cloudinary:

```
ts
KopierRediger
// server/storage.ts
import { v2 as cloudinary } from "cloudinary";
cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
```

```

    api_key: process.env.CLOUDINARY_API_KEY,
    api_secret: process.env.CLOUDINARY_API_SECRET,
  });

export async function uploadToCloudinary(filePath: string) {
  return cloudinary.uploader.upload(filePath, { folder: "SELFIE_AI"
});
}

```

■

For Mux, create `server/muxRoutes.ts` if users upload videos:

```

ts
KopierRediger
// server/muxRoutes.ts
import Mux from "@mux/mux-node";
const { Video } = new Mux(process.env.MUX_ENV_KEY,
process.env.MUX_ENV_SECRET);

export async function createMuxAsset(req, res) {
  const { uploadUrl } = req.body;
  const asset = await Video.Assets.create({ input: uploadUrl });
  res.json(asset);
}

```

■

Register route in `server/routes.ts`:

```

ts
KopierRediger
import { createMuxAsset } from "../muxRoutes";
router.post("/api/mux/createAsset", createMuxAsset);

```

■

3. Advanced Image & AR Filters (Banuba or YouCam)

- **Frontend:**

In `client/client/src/components/Studios/VisualsStudio.tsx`, install and import Banuba or YouCam React SDK:

```
tsx
KopierRediger
import BanubaSDK from "react-banuba";
<BanubaSDK
  licenseKey={process.env.BANUBA_LICENSE_KEY}
  onRenderFrame={(frame) => { /* render to Konva canvas */ }}
/>
```

-
- Add dropdown or buttons for “Golden Crown” and “Diamond Glow” effects, using Banuba’s effect IDs.
- **Backend:**
 - None required unless storing custom AR assets.

4. Sentiment & Emotional Intelligence (Google NL or AWS Comprehend)

- **Backend:**

Create `server/sentiment.ts`:

```
ts
KopierRediger
import language from "@google-cloud/language";

const client = new language.LanguageServiceClient();

export async function analyzeSentiment(text: string) {
  const [result] = await client.analyzeSentiment({
    document: { content: text, type: "PLAIN_TEXT" },
  });
  return result.documentSentiment; // { score, magnitude }
}
```

■

In `server/sandraChatAI.ts`, after transcription, call `analyzeSentiment(transcript)`.
If `score < 0`, prepend more empathetic intro to the prompt, e.g.:

```
ts
KopierRediger
const sentiment = await analyzeSentiment(transcript);
let pepPrompt = `User said: "${transcript}"`;
if (sentiment.score < 0) {
  pepPrompt = "I sense you're feeling frustrated—here's a mini pep
talk:\n\n" + pepPrompt;
}
const finalPrompt = formatWithSandraTone(pepPrompt + " Provide a quick
encouragement and one actionable prompt.");
```

5. Voice & Speech Processing (Google Speech-to-Text or AWS Transcribe, Twilio SMS)

Backend (Google STT example):

```
ts
KopierRediger
// server/speechToText.ts
import speech from "@google-cloud/speech";
const client = new speech.SpeechClient();

export async function transcribeAudio(buffer: Buffer): Promise<string>
{
  const [response] = await client.recognize({
    audio: { content: buffer.toString("base64") },
    config: { encoding: "LINEAR16", languageCode: "en-US" },
  });
  return response.results.map(r =>
r.alternatives[0].transcript).join("\n");
}
```

Twilio SMS Nudges:

```
ts
KopierRediger
// server/smsNudge.ts
import twilio from "twilio";
const client = twilio(process.env.TWILIO_SID,
process.env.TWILIO_TOKEN);

export async function sendDailyPromptSMS(userPhone: string, prompt:
string) {
  return client.messages.create({ from: process.env.TWILIO_PHONE, to:
userPhone, body: prompt });
}
```

○

6. Email Marketing & Automations (Flodesk or Mailchimp, Zapier/Make)

Backend (Mailchimp example):

```
ts
KopierRediger
// server/email/workflows.ts
import mailchimp from "@mailchimp/mailchimp_marketing";

mailchimp.setConfig({
  apiKey: process.env.MAILCHIMP_API_KEY,
  server: process.env.MAILCHIMP_SERVER_PREFIX,
});

export async function addTo30DayChallenge(email: string) {
  await mailchimp.lists.addListMember(process.env.MAILCHIMP_LIST_ID, {
    email_address: email,
    status: "subscribed",
  });
}
```

○

- In your StarterKit signup flow (e.g., in `StarterKit.tsx` after payment), call `/api/email/addTo30DayChallenge`.

- **Zapier/Make:** Set up a webhook endpoint in `server/webhooks.ts` triggered when a user completes Day 5 of a workbook. Configure a Zap to invite them to a private Slack or Discord channel.

7. Push Notifications & Real-Time Chat (OneSignal, Pusher, Firebase).

Frontend (OneSignal):

```
tsx
KopierRediger
// client/client/src/index.tsx (or main entry)
import OneSignal from "react-onesignal";

OneSignal.init({ appId: process.env.ONE_SIGNAL_APP_ID });
OneSignal.sendTag("userId", "CURRENT_USER_ID");
```

○

In components (e.g., `MonthlyWorkbook.tsx`, `PostScheduler.tsx`), send push notifications:

```
ts
KopierRediger
OneSignal.sendSelfNotification("Your new AI Workbook just dropped!",
"Log in to see your 10-day micro-challenge!");
```

■

Real-time chat with Pusher (in `SisterCircle.tsx`):

```
tsx
KopierRediger
import Pusher from "pusher-js";

const pusher = new Pusher(process.env.PUSHER_KEY as string, { cluster:
process.env.PUSHER_CLUSTER });
const channel = pusher.subscribe(`pod-${podId}`);
channel.bind("new-message", (data: any) => {
  setMessages(prev => [...prev, data]);
});
```

○

Backend (server/notifications.ts):

```
ts
KopierRediger
// server/notifications.ts
import OneSignalClient from "onesignal-node";
import { getDbClient } from "../db";

const client = new OneSignalClient(process.env.ONE_SIGNAL_APP_ID,
process.env.ONE_SIGNAL_API_KEY);

export async function sendNotification(userId: string, title: string,
message: string) {
  const db = getDbClient();
  const userRes = await db.query("SELECT id FROM users WHERE id = $1",
[userId]);
  if (!userRes.rows.length) return;
  await client.createNotification({
    headings: { en: title },
    contents: { en: message },
    include_external_user_ids: [String(userId)],
  });
}
```

-
- If using Firebase for live chat, you can replace Pusher logic with Firebase Realtime Database or Firestore listeners.

8. Payments & Subscriptions (PayPal, Paddle, Chargebee, Recurly).

Frontend (PayPal example in Pricing.tsx):

```
html
KopierRediger
<div id="paypal-button-container"></div>
<script
src="https://www.paypal.com/sdk/js?client-id=YOUR_PAYPAL_CLIENT_ID&cur
rency=USD"></script>
<script>
  paypal.Buttons({
```

```

    createOrder: function (data, actions) {
      return actions.order.create({
        purchase_units: [{ amount: { value: "47.00" } }]
      });
    },
    onApprove: function (data, actions) {
      return actions.order.capture().then(function (details) {
        fetch("/api/paymentHandlers/paypal", {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ orderID: data.orderID, userId:
"CURRENT_USER_ID" })
        });
      });
    }
  }).render("#paypal-button-container");
</script>

```

○

Frontend (Paddle example):

```

html
KopierRediger
<script src="https://cdn.paddle.com/paddle/paddle.js"></script>
<script>
  Paddle.Setup({ vendor: YOUR_VENDOR_ID });
  function openPaddleCheckout() {
    Paddle.Checkout.open({ product: YOUR_PRODUCT_ID, email:
"user@example.com" });
  }
</script>
<button onClick="openPaddleCheckout()">Buy SELFIE PRO via
Paddle</button>

```

○

Backend (server/paymentHandlers.ts – PayPal example):

ts

KopierRediger

```
// server/paymentHandlers.ts
import { Router } from "express";
import PayPal from "@paypal/checkout-server-sdk";
import { grantSubscription } from "../db";

const router = Router();
const client = new PayPal.core.PayPalHttpClient(
  new PayPal.core.SandboxEnvironment(process.env.PAYPAL_CLIENT_ID,
process.env.PAYPAL_SECRET)
);

router.post("/api/paymentHandlers/paypal", async (req, res) => {
  const { orderId, userId } = req.body;
  const request = new PayPal.orders.OrdersGetRequest(orderId);
  const order = await client.execute(request);
  if (order.result.status === "COMPLETED") {
    await grantSubscription(userId, "SELFIE_PRO", order.result.id);
    return res.json({ success: true });
  }
  return res.status(400).json({ error: "Payment not completed." });
});

export default router;
```

-
- If you later need advanced billing (region pricing, enterprise), integrate Chargebee or Recurly's Node SDK in a similar handler.

9. Search & Recommendation Engine (Algolia or Elasticsearch).

Frontend (**client/client/src/algolia.js**):

```
js
KopierRediger
import algoliasearch from "algoliasearch";

const client = algoliasearch("YOUR_ALGOLIA_APP_ID",
"YOUR_ALGOLIA_API_KEY");
```

```
export const resourceIndex = client.initIndex("selfie_resources");
```

-
- In `ResourceLibrary.tsx`, use `resourceIndex.search(searchQuery)` to populate search results.
- **Backend (`server/resources.ts`):**

After any resource insertion/update, push to Algolia:

```
ts
KopierRediger
import algoliasearch from "algoliasearch";

const algoliaClient = algoliasearch(process.env.ALGOLIA_APP_ID,
process.env.ALGOLIA_API_KEY);
const index = algoliaClient.initIndex("selfie_resources");

export async function syncResourceToAlgolia(resource) {
  await index.saveObject({
    objectID: resource.id,
    title: resource.title,
    description: resource.description,
    link: resource.link,
    tags: resource.tags,
  });
}
```



10. Calendar & Event Integrations (Google Calendar API, Calendly).

Frontend (Google Calendar integration in `PostScheduler.tsx`):

```
tsx
KopierRediger
// Add this inside PostScheduler, next to the "Schedule Post" button
const addToCalendar = async () => {
  await fetch("/api/calendar/createEvent", {
    method: "POST",
```

```

        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ userId: "CURRENT_USER_ID", content,
datetime: date.toISOString() }),
    });
};

```

```

// In JSX:
<button onClick={addToCalendar} className="ml-2 bg-green-500
text-white p-2 rounded">
    Add to Google Calendar
</button>

```

○

Frontend (Calendly in CoachingApplication.tsx):

```

tsx
KopierRediger
// client/client/src/pages/CoachingApplication.tsx
export default function CoachingApplication() {
    return (
        <div className="p-4">
            <h2>Apply for Empire VIP Coaching ($15K/3 months)</h2>
            <iframe
                src="https://calendly.com/your-calendly-link"
                width="100%"
                height="600"
                frameBorder="0"
            />
        </div>
    );
}

```

○

Backend (server/calendar.ts):

```

ts
KopierRediger
// server/calendar.ts
import { google } from "googleapis";

```

```

// OAuth2 client setup
const oauth2Client = new google.auth.OAuth2(
  process.env.GOOGLE_CLIENT_ID,
  process.env.GOOGLE_CLIENT_SECRET
);
oauth2Client.setCredentials({ refresh_token:
process.env.GOOGLE_REFRESH_TOKEN });

export async function createCalendarEvent(req, res) {
  const { userId, content, datetime } = req.body;
  const event = {
    summary: `SELFIE AI™ Post: ${content.slice(0, 20)}...`,
    start: { dateTime: datetime },
    end: { dateTime: new Date(new Date(datetime).getTime() + 5 * 60 *
1000).toISOString() },
  };
  try {
    await google.calendar("v3").events.insert({
      calendarId: "primary",
      resource: event,
      auth: oauth2Client,
    });
    res.json({ success: true });
  } catch (error) {
    res.status(500).json({ error: "Failed to create calendar event."
});
  }
}

```

○

Register route in `server/routes.ts`:

```

ts
KopierRediger
import { createCalendarEvent } from "../calendar";
router.post("/api/calendar/createEvent", createCalendarEvent);

```

■

10. TESTING & QA

1. Run lint & tests after each major change.

In terminal, project root:

```
bash
KopierRediger
# Frontend
cd client/client && npm run lint && npm run test

# Backend
cd server && npm run lint && npm run test
```

-
- Fix all errors before merging branches.

2. Manual core flow tests:

- **Sign-up → Starter Kit → PRO Onboarding**
 - Register a new user; verify Supabase auth (login, session persistence).
 - Visit `/starter-kit`, generate all 3 workbooks.
 - Purchase SELFIE PRO (\$47/mo) via Stripe or PayPal; confirm subscription and PRO access.
- **Post Scheduling → Calendar Sync**
 - Create a post in `PostScheduler`, pick a time, schedule it.
 - Click “Add to Google Calendar,” verify event appears in your Google Calendar.
- **Monthly Workbook → Voice Check-in → Sister Circle → Content Hacks**
 - As a PRO user, navigate to `/modules/Pro/Workbooks/MonthlyWorkbook`, check tasks, mark

complete.

- In `/components/Pro/VoiceCheckIn`, upload a short audio file; verify micro-prompt output.
- In `/modules/Pro/SisterCircle`, send and receive messages between two test accounts.
- In `/modules/Pro/ContentHacks`, verify a 3-line script is generated and copy-paste works.

- **Premium Add-Ons**

- In `/components/BrandKitGenerator`, input sample data, verify logo URL, palette, font preview, moodboard link.
- In `/modules/ResourceLibrary`, complete quiz, check top 5 resources appear; test search bar with a keyword.
- In `/components/BrandThermometer`, click “No,” fill micro-module, verify action steps.
- Manually trigger spotlight cron: `node server/cron/spotlightSuccess.ts`; verify `/public/spotlights/spotlight-<userId>.html` is created.
- Manually trigger monthly audit digest: `node server/cron/auditDigest.ts`; verify sendEmail logs or SendGrid dashboard.

3. Review Git diffs before merging.

- Use GitHub Pull Requests or run `git diff main...feature-branch` locally.
- Ensure only intended files are added/modified/deleted—no accidental deletions in `public/` or `.env` changes.

4. Document test results in `CHANGELOG.md`.

In repo root, maintain `CHANGELOG.md` with entries such as:

```
md
KopierRediger
## [2025-06-06] – v1.3.0
### Added
- `StoryArcBuilder.tsx` (3-step story arc generator)
- Starter Kit module with 3 AI workbooks
- Branded Lab components: `BrandPlanWorkbook.tsx`,
  `BrandAssetsGenerator.tsx`
- PRO modules: `MonthlyWorkbook.tsx`, `VoiceCheckIn.tsx`,
  `SisterCircle.tsx`, `ContentHacks.tsx`
- `ContentStarter.tsx` for one-click prompts
- Dashboard metrics endpoints: `getConfidenceScore`,
  `getActionStreak`, `hasCompletedStarterKit`
- Premium add-ons: `BrandKitGenerator.tsx`, `ResourceLibrary.tsx`,
  `SwipeVault.tsx`, `BrandThermometer.tsx`
### Changed
- Refactored `VisualsStudio.tsx` (fixed mobile swipe & save)
- Removed `CaptionsStudio.tsx` and old 6-step caption logic
- Updated AI endpoints in `server/storyArcAI.ts` (3-step arc)
### Fixed
- Bug: Edited images now save correctly in `VisualsStudio.tsx`
- Bug: Enforced 20+ character validation in `StoryArcBuilder.tsx`
  ○ - Cron jobs: `auditDigest.ts`, `spotlightSuccess.ts`
```